# HStore: A Copy-on-Write Storage Engine for Hypergraphs

Metacog Labs
Metacog Labs
India

## ABSTRACT

A hyperedge relates any number of things at once, and many datasets are most naturally hypergraphs. Graph databases store them by reification, which hides each relation's size, members and identity from the storage engine. We describe HStore, an embedded storage engine built for hyperedges. Every structure in HStore, including each hyperedge's membership and the index from an atom to the hyperedges that contain it, is a copy-on-write B+tree over 64-bit keys whose child references carry entry counts and subtree summaries. Small memberships are stored inside the hyperedge's record. Commits log the keys they changed instead of the pages they rewrote, a background checkpoint writes each changed node once, and recovery always rebuilds a prefix of the commit order. Pages are reached through a page directory, so compaction moves them without rewriting any tree, generation or branch. We compare HStore with HyperGraphDB on 26 workloads with asynchronous and synchronous commits and with data larger than both systems' caches. HStore reads incidence sets 2.4 to 2.5 times faster, writes 224 B per update against 9.12 KiB, and ingests hyperedges 7.9 times faster once the data outgrows the cache. Its median single-commit latency under asynchronous durability is the one measurement where HyperGraphDB is ahead.

## 1 INTRODUCTION

Many datasets are made of relations among more than two things. A medical claim involves a patient, a provider, a payer and a list of procedures. A paper has authors in a fixed order. A reaction has reactants, products and a catalyst, each in its own role. A hypergraph models each such relation as one *hyperedge* that contains any number of members, where a member can play a role, carry a weight or hold only for a period of time. If hyperedges are objects in their own right, a hyperedge can also be a member of another one. Statements about statements, provenance and evidence are expressed this way [2, 30].

Graph databases store binary edges. The usual way to store an $n$-ary relation in one is reification: a node stands for the relation and $n$ binary edges connect it to its members [1]. Queries still work, but the storage engine now sees $n + 1$ unrelated objects. It cannot say how many members a relation has without visiting all $n$ edges or compare two relations without reading both, and a change to a membership becomes $n$ separate edge updates that a concurrent writer cannot merge as a unit.

Several systems model hyperedges directly. HyperGraphDB [14] stores each link as an array of target handles in Berkeley DB Java Edition [20], with a separate index from every atom to the links that point at it. AtomSpace [11] and TypeDB [9] make $n$-ary relations first class in their data models, and parallel frameworks [12, 13, 28] analyse hypergraphs held in memory. What is still missing is a storage engine whose physical structures are designed around hyperedges. Such an engine should make small and very large memberships equally cheap to read and change, answer membership, cardinality and overlap questions without scanning, let writers commit concurrently while readers see stable snapshots and past versions, and recover from a crash to a state that some prefix of the commits produced.

This paper describes HStore, an embedded storage engine built to those requirements, and compares it with HyperGraphDB. Four ideas shape it.

*One tree for every structure.* Every structure HStore keeps is a copy-on-write B+tree over 64-bit keys, including each hyperedge's membership and the reverse index from an atom to the hyperedges that contain it. Each child reference in the tree carries the entry count and a summary of its subtree: the key range, the weight and time ranges, a role mask and a fingerprint that depends only on the subtree's contents. Counts give rank and select in logarithmic time. Summaries let a query skip subtrees it cannot match, and let two trees be compared, intersected or diffed in time that grows with how much they differ rather than with their size. The catalog is a tree of trees: a hyperedge's record holds the root of its membership tree, and holds the whole tree when it is small, as almost all are.

*Commits log keys, not pages.* A copy-on-write commit normally writes every node from each changed leaf up to the root. An HStore commit logs the keys it changed and publishes a generation whose new nodes are still in memory. A background checkpoint writes each of those nodes once, however many commits changed it. Recovery replays the logged keys from the last checkpoint in commit order and stops at the first commit whose pages did not reach the disk, so the recovered state is always a prefix of the commit order.

*Pages move without rewriting trees.* Trees refer to pages by permanent page numbers, and a memory-mapped page directory maps each number to the page's current place in an append-only segment file. Compaction copies a live page byte for byte and changes one directory entry. No tree, generation or branch is rewritten, so compaction does not conflict with commits and keeps old generations readable. Reference counts, updated only between consecutive checkpointed generations, tell compaction which pages are live.

*Leaves decode one record at a time.* Records in catalog and index leaves are prefixed by their length. A leaf read from disk keeps its bytes and decodes a record only when the record is first read, charging the decoded size to the node cache. When the data is larger than the cache, a lookup then decodes one record instead of a whole page.

HStore is written in Java 25 and consists of the storage engine, a database layer with schemas, properties and a query language, and a server. We compare it with HyperGraphDB on a synthetic hypergraph with skewed degrees, using 26 workloads that cover ingest, reads, per-operation latency, reads during writes, deletes,

space reclamation and crash recovery. We run them with asynchronous and synchronous commits, and with a dataset larger than both systems' caches. HStore is ahead on every workload whose rate the harness does not fix, with one exception: the median latency of a single commit under asynchronous durability, where HyperGraphDB is faster. HStore writes 224 B per update where HyperGraphDB writes 9.12 KiB, and its files after the load are 4.7 to 13.7 times smaller. It keeps more of the Java heap than HyperGraphDB does.

Section 2 defines the data model. Section 3 gives an overview of the engine, and Sections 4 to 6 describe the storage layout, transactions and recovery, and space reclamation. Section 7 covers the implementation and its testing, and Section 8 the evaluation. Section 9 discusses related work, and Section 10 concludes with the limitations we know of.

## 2 DATA MODEL

*Atoms.* Everything HStore stores is an *atom* with a 64-bit identifier, handed out in increasing order. An atom is a *node* or a *hyperedge*. Both have a type, both can have properties, and a node can also have a canonical key that is unique within its type, such as `Person:'alice'`.

*Hyperedges and incidences.* A hyperedge has any number of members. Each membership is an *incidence*

$$(\mathit{member}, \mathit{roles}, \mathit{weight}, [\mathit{from}, \mathit{to}), \mathit{data}, \mathit{qualifier}),$$

where *member* is any atom, including another hyperedge; *roles* is a set of role names (a member can be both buyer and approver); *weight* is an exact fixed-point number with nine decimal places; $[\mathit{from}, \mathit{to})$ is the interval in which the membership holds; *data* points to a payload and *qualifier* to an atom that records evidence for it. All fields except the member have defaults. A *set* hyperedge holds each atom at most once and has no order. An *ordered* hyperedge is a sequence that can be edited at any position, and the same atom can appear in it only once.

Figure 1 shows one hyperedge of each kind from the example dataset that ships with HStore.

*Generations, history and branches.* Every commit produces a new *generation*, numbered consecutively. A reader opens a snapshot of a generation and sees it unchanged for as long as it keeps it open. The engine keeps a configurable number of recent generations (64 by default) for time travel, which can be addressed by number or by wall-clock time. A *branch* is a named line of generations forked from another branch. Creating one copies nothing, and the database layer merges branches with a three-way merge against the fork point.

*Operations.* Writes create and delete atoms, change their records, and change memberships: add, update or remove a member by atom, insert, update or remove at a position in an ordered hyperedge, and replace a whole membership in one bulk load. Deleting an atom removes it from every hyperedge that contains it. Reads look up an atom, test and fetch a member, return a hyperedge's cardinality and the member at a given rank, scan members filtered by role, weight or validity time, list the hyperedges incident to an atom, intersect incidence sets ("hyperedges that contain all of $a_1, \ldots, a_k$"), and compute differences between two hyperedges, two generations or two branches.

*Layers.* The storage engine knows only atoms, memberships and a set of keyed trees. Types, property values, tenants and users, the HQL query language, an embedding index and the higher-order operators are added by a database layer that stores its own data in additional trees inside the same generations (Section 7). The rest of this paper is about the engine.

## 3 OVERVIEW

Figure 2 shows the main parts of the engine. The state of the database at one point in time is a *generation*: a small immutable object that holds, for each branch, a *root vector* that maps a slot number to the root of a tree. The engine defines seven slots: the *catalog* of atoms, the reverse incidence index, an index from type to atoms, an index of canonical keys, two symbol tables for role names, and a table of request ids that makes retried commits idempotent. The database layer adds its own slots for schemas, properties, property indexes and the like. Slots are either primary, written by transactions, or derived, maintained from the primary slots inside the same transaction by *derivations*. The reverse incidence index is derived from the catalog.

All trees are copy-on-write. A transaction works on a private copy of the root vector, and its writes copy the paths they change. A commit publishes a new generation that shares every unchanged subtree with its predecessor. Readers pin a generation and never take a lock. Writers serialise only on a short section of the commit that assigns the generation number and appends to the log, and commits that reach the log while an earlier batch is being synced share one fsync.

The tree nodes a commit creates are not written at commit time. The commit logs the keys it changed and publishes nodes that are still in memory; a background checkpoint later writes them to pages. Pages are packed into append-only segment files and found through a page directory, so that compaction can move them without changing any tree. The rest of the paper follows this structure: trees and the on-disk layout (Section 4), commits, checkpoints and recovery (Section 5), and reclaiming space (Section 6).

## 4 TREES AND STORAGE LAYOUT

### 4.1 Counted, summarised copy-on-write trees

Every tree in HStore is a B+tree over signed 64-bit keys. A leaf holds sorted keys and their values; a branch holds separator keys and child references. Nodes are never changed once another version can see them. An update copies the path from the root to the changed leaf, and the new root shares every other subtree with the old one, as in other copy-on-write B-trees [5, 18, 25]. Inside one transaction, a node that the transaction itself created may be changed in place: each node carries an owner token, and a write copies a node only if its token is not the transaction's. A transaction that makes $n$ changes under one root therefore allocates in proportion to $n$ plus the nodes it touches, not $n$ times the tree height. When a version escapes, for example to a stream handed to the caller, the token is replaced and every node created so far becomes immutable.

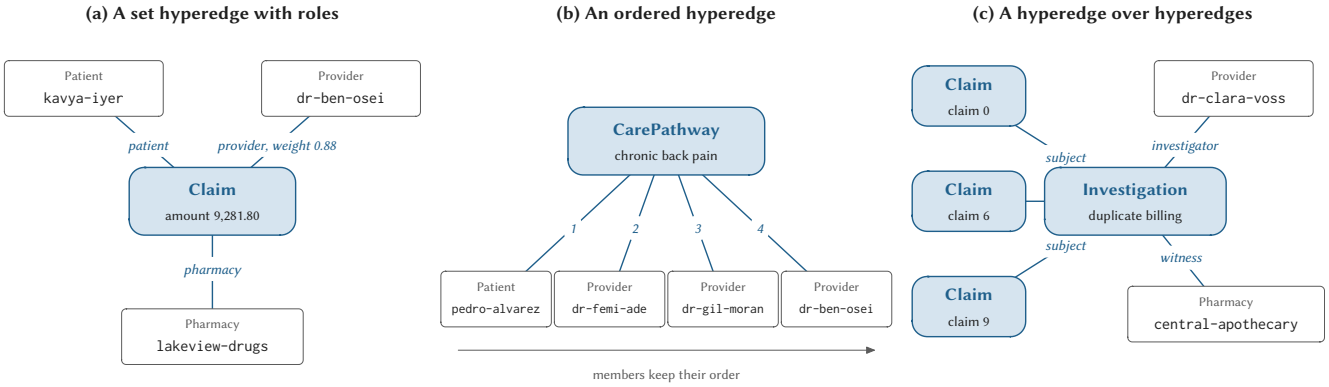



**Figure 1: Three hyperedges from the clinical claims example that ships with HStore. (a) A claim relates a patient, a provider and a pharmacy, each in its own role; the provider's membership carries a weight. (b) A care pathway lists a patient and the providers who saw them, in order. (c) An investigation has three claims as its subjects, so its members are themselves hyperedges.**

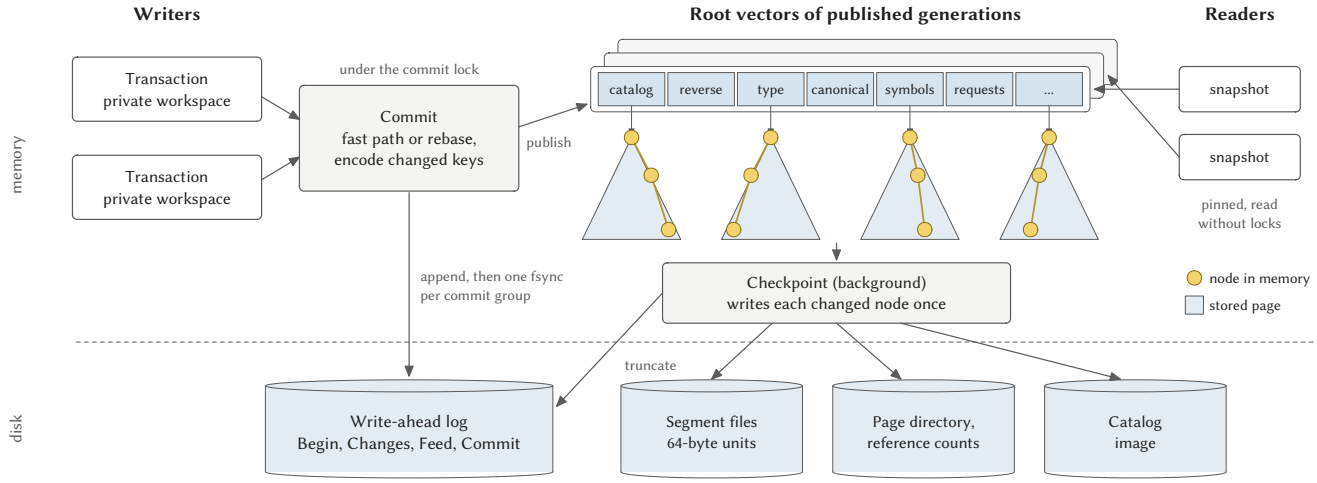



**Figure 2: Engine overview. A commit appends the keys it changed to the write-ahead log and publishes a new root vector whose changed nodes (yellow) are still in memory, on top of stored pages (blue). Readers pin a generation and read it without locks. A background checkpoint writes the changed nodes, syncs them, publishes a catalog image and truncates the log.**

Leaves are filled by bytes, not by entry count. A leaf tracks the exact size of its delta-encoded key stream and an upper bound on the size of its values, and splits when the total exceeds the page budget. With the default 16 KiB page, a leaf of plain 8-byte keys holds thousands of entries and a branch holds up to 137 children.

*Summaries.* Each child reference stores the subtree's *summary*, a monoid value made of the entry count, the minimum and maximum key, the sum and range of the entry weights, the range of the validity intervals, a 64-bit mask of the role sets present, and a fingerprint. Combining two summaries is component-wise: sums, minima, maxima and bitwise or. The count makes rank, select and range counts logarithmic, since they subtract or add child counts from the references without loading the children. The other fields let a cursor skip any subtree that cannot hold a match: a scan for members valid at time $t$ enters a child only if $t$ lies in the child's time range, and a scan by role only if the role's bit is set in the mask. The mask is a 64-bucket Bloom filter, so a set bit only means "maybe" and the scan checks the role exactly.

*Fingerprints.* Let $h(e)$ be a 64-bit hash of an entry. A tree whose order does not matter, such as the members of a set hyperedge, has the fingerprint $F(S) = \sum_{e \in S} h(e) \bmod 2^{64}$, an additive multiset hash [6]. A sequence uses a polynomial hash [15], $F(e_1 \dots e_n) = \sum_i h(e_i)\, B^{n-i} \bmod 2^{64}$, whose concatenation rule $F(LR) = F(L)\, B^{|R|} + F(R)$ lets a branch combine its children's

fingerprints. Both depend only on the contents, never on the shape of the tree or the order of insertion. Two hyperedges with different fingerprints differ, and the engine knows it after reading two references. All hashes are computed from explicit mixing functions, not from Java's object hash codes, so fingerprints stay the same across processes and across the native build.

*Algebra on shared structure.* Because versions share subtrees, comparing two of them can skip whatever they share. Each reference to a stored subtree is identified by its page number, and two equal references denote the same subtree. A diff walks two frontiers of subtrees and drops a pair of equal references without reading it, so the diff of two consecutive generations costs about twice the tree height in page reads. Counting the intersection of two trees descends both in step: disjoint key ranges contribute nothing, equal references contribute their stored count, and only the parts that differ are read. Intersecting $k$ incidence sets uses a leapfrog join [29] over $k$ cursors, each restricted by summaries to the key range common to all inputs. A union of two trees with disjoint key ranges is a concatenation that copies $O(1+\Delta h)$ nodes, where $\Delta h$ is the difference in height.

## 4.2 Hyperedges

The members of a set hyperedge are a tree keyed by member atom id, whose values are incidences. Membership tests and the member at rank $i$ are logarithmic, and the cardinality is the count in the root reference.

An ordered hyperedge is a sequence that can be edited at any position, so its keys are *order-maintenance labels* [3, 8]: sparse integers in $[0, 2^{62})$ whose order is the sequence order. The incidence stores the member id. An insertion between two members takes the midpoint of their labels; an append steps by $2^{24}$, so appending never relabels. When two neighbours have no free label between them, the engine relabels a window around the position, doubling its width until the free space reaches $2^8$ labels per member, which keeps the amortised cost of an insertion logarithmic. A second tree maps each member to its label, so membership tests stay logarithmic. Relabelling changes keys but not contents, so the fingerprint of an ordered hyperedge, which is computed from the incidences and not from their labels, does not change.

Incidence leaves are encoded column by column. A flag byte says which of the six optional columns (roles, weight, start and end of validity, payload, qualifier) any row in the leaf uses. Each such column has a bitmap of the rows that use it, followed by their values as variable-length integers. A plain membership therefore costs one to three bytes, and a leaf whose rows use no optional column is decoded into nothing but an array of member ids.

## 4.3 The catalog: a tree of trees

The catalog maps an atom id to its record (Figure 3a). A hyperedge's record holds references to the root of its membership tree and, for ordered hyperedges, of its label index. These are nested trees that the engine writes, counts and compacts like any other.

Most hyperedges are small: in the dataset of Section 8, the mean cardinality is 3.6. Giving each membership a page of its own would cost a page write, a cache entry and a page read for every hyperedge read. A nested tree is therefore kept *inline* in its record when it is a single leaf of at most 32 entries whose encoding fits in 108 bytes, the space a record already reserves for a reference to a tree that is still in memory. That rule keeps pages from overflowing. While a hyperedge is being changed, the leaf that holds its record is sized as if the reference took the full 108 bytes. When the leaf is written, the reference becomes either a stored or an inline one, both of which fit in that space, so the written leaf can only shrink. The decision is made again each time the tree is written, so a hyperedge that grows past the limit moves to its own pages at the next checkpoint and one that shrinks moves back.

## 4.4 The reverse incidence index

The reverse index maps an atom to the hyperedges that contain it (Figure 3b). It is a two-level structure. A directory tree maps the atom to a posting set; a posting set maps a hyperedge id to the member's roles and its position key in that hyperedge. A posting set is stored inline in the directory leaf while it has at most 48 entries and at most half the largest value size, and is promoted to a tree of its own beyond that. It is demoted again when it shrinks to 24 entries, so a set that hovers at the limit does not move back and forth. The directory's summaries add up posting counts as weights, so the number of incidences of any range of atoms is read from the summaries alone. The index is a derived slot: a derivation applies every membership change in a transaction to it before the transaction commits, so the two directions always agree within a generation.

## 4.5 Node images, page numbers and segments

A node is stored as an *image*: an 80-byte header followed by the encoded node. The header holds a magic number, the page type, the tree's schema id, the node's own page number, the generation that wrote it, the payload length, the entry count, the node's summary and a CRC-32C over everything except the checksum itself. Every read verifies the magic number, the format, the page number and the checksum.

A page number is permanent. It has a 48-bit index and 16 bits reserved for reuse, and it says nothing about where the image is stored. The *page directory* maps each index to an address made of a 24-bit segment number, a 24-bit offset in 64-byte units and a 16-bit length in units. The directory is a file of 8-byte entries, memory-mapped in 8 MiB chunks, so a lookup is one load and takes no Java heap. Trees, the node cache and the log refer only to page numbers; only the page store resolves addresses. Page `#1234`, stored in three units at unit 1234 of segment 1, is the directory entry at byte $64 + 8 \cdot 1234$, pointing to byte $64 \cdot 1234$ of segment file `00000001.seg`.

Images are packed back to back into append-only segment files and aligned to 64 bytes, rather than given a fixed slot of the page size. A hypergraph produces many small nodes, and on the example database that ships with HStore packing cuts the segment space from 17.6 MB to 1.0 MB. Each segment is mapped read-only, and a read copies the image out of the map into a heap array before decoding it. Decoding straight from the map was two to five times slower in a microbenchmark, because the JIT compiler generates its fastest code only when every byte buffer that reaches the decoder is of the same kind, and the log and the catalog are decoded from heap arrays.

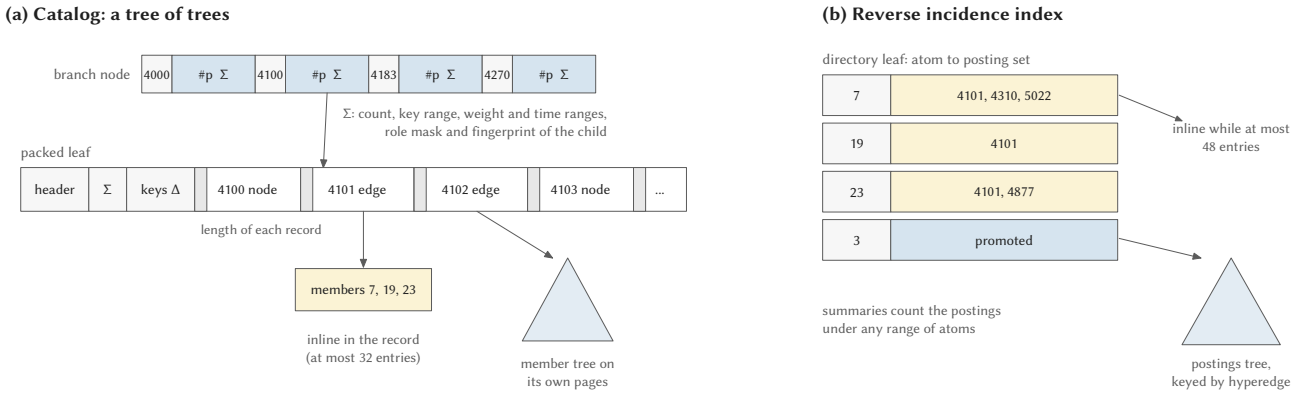



**Figure 3: (a) The catalog holds atom records. A hyperedge record holds its membership tree inline when the tree is one small leaf and points to its own pages otherwise. Every record in a catalog leaf is preceded by its length. (b) The reverse incidence index maps an atom to the hyperedges that contain it. Small posting sets are stored in the directory leaf, large ones in a tree of their own.**

## 4.6 Packed leaves and the node cache

Decoded nodes are cached on the Java heap in a node cache with 16 shards. Each shard is an open-addressing table keyed by page number with a CLOCK ring [7] for eviction and a byte budget. Lookups are lock-free; only admission takes the shard's lock. A node is charged an estimate of its heap size, computed from its arrays and entries with constants measured against the retained heap at two dataset sizes.

Catalog and index leaves hold variable-length records, and a fully decoded catalog leaf takes far more heap than its page. On our dataset a decoded catalog leaf of about 83 records, with their inline member lists, took about eleven times the size of its 1.5 KiB image. With a cache smaller than the catalog, lookups missed the cache about half the time and each miss decoded every record of the page to return one. Leaves of codecs whose values are independent of each other therefore prefix every record with its length. A leaf read from a page is kept *packed*: it holds the image, the decoded keys and the offset of every record, and decodes a record the first time it is read. The decoded record is published into the leaf with a compare-and-set, so concurrent readers see a whole record and keep whichever decoding finished first, and its estimated size is charged to the leaf's cache slot. A charge evicts other entries until the shard is back within its budget, and a charge for a slot that was evicted in the meantime is dropped. The length prefix costs about one byte per record; Section 8.6 measures what it buys.

# 5 TRANSACTIONS AND DURABILITY

## 5.1 Snapshots and isolation

The transaction manager keeps two generations: *head*, the newest generation appended to the log, and *current*, the newest one that is durable and published. New readers and transactions pin *current*, so no one sees a commit that a crash could still lose. Pins are reference counts per generation. Old generations are trimmed to the history limit, except that a pinned generation is skipped rather than treated as a barrier, so one long-running reader keeps only its own generation alive. Reading an old generation costs the same as reading the newest one: copy-on-write never changes a page that a retained generation can reach, so time travel needs no undo log.

Transactions run under snapshot isolation by default [4]. Under the serializable level, a transaction records the value of every key and atom it reads as of its base generation, and a commit that has to rebase (below) checks that none of them changed. This rejects write skew, at key granularity and only when a slot the transaction read has moved.

## 5.2 Commit

A transaction keeps an *operation log*: every operation it applied, stored as an intent such as "insert member $m$ into hyperedge $e$" or "update key $k$ of slot $s$ if its value is still $v$". The operations are also applied immediately to the transaction's private copy of the trees, and the derivations update the derived slots in the same copy.

At commit, the transaction takes the commit lock and compares its base root vector with *head*. If no slot it changed has moved, it takes the fast path: the new root vector is *head*'s with the transaction's changed slots laid over it. Commits that write different slots therefore never wait for each other, however many commits land in between. Otherwise the commit replays its operation log against *head*. Replay fails with a retryable conflict when a precondition no longer holds, with one exception that matters for hypergraphs. When the only change to a set hyperedge was other transactions adding or removing members, and every action of this transaction on it named a member, the commit diffs the hyperedge's old and new membership trees and conflicts only if a changed member is one it touched. Two transactions that add different members to the same hyperedge therefore both commit. Positional actions on ordered hyperedges and bulk loads conflict whenever the hyperedge moved. A commit can carry a request id. It is recorded in a slot under the commit lock, so a retried commit returns the original generation instead of applying twice.

## 5.3 Logging keys instead of pages

A copy-on-write commit that writes its new pages writes every node from each changed leaf up to the root of each changed tree. For one updated property, that is a few leaves and branches in the property tree, its index and the catalog, many kilobytes for a few bytes of change. HStore commits instead log the keys they changed and publish a generation whose new nodes are still in memory.

The write-ahead log is a sequence of segment files addressed by a log sequence number (LSN), the byte offset since the log was created. Every record is a frame with a 25-byte header that holds the payload length, a CRC-32C, the frame's own LSN, the record type and the transaction id. Because the checksum covers the LSN, stale bytes left in a reused file can never pass for a valid frame. A commit appends, under the commit lock, a `Begin` record, one `Changes` record, a `Feed` record with the commit's logical events for the change feed, and then, in a separate append, the `Commit` record with the generation number and the next free atom and transaction ids. The `Commit` frame is the commit point.

The `Changes` payload lists, for each slot whose root changed, the keys that differ between the base tree and the new one, found with the diff of Section 4.1. A changed value that holds nested trees, such as a hyperedge record, is followed by the changes to each nested tree, so a commit that adds one member to a hyperedge logs that member, not the hyperedge. A tree that changed in more than 4,096 places, as after a large bulk load, is written to pages at commit and logged by reference instead. Each such page gets a 47-byte `PageRef` record with its page number, address, length and CRC-32C, and the data files are forced before the log whenever a commit group contains such pages. Large bulk loads also *spill* finished subtrees to pages while they run, so that a hyperedge with millions of members never has to fit in memory.

Commits do not wait for the disk under the commit lock. The lock is released after the `Commit` frame is appended, and the committing thread then waits on a group committer. The group committer has no thread of its own: the first waiting committer becomes the leader, takes every queued commit as one batch, syncs once, publishes the batch in generation order and wakes the others. Under synchronous durability the batch forces the data files if any of its commits wrote pages and then the log; under asynchronous durability nothing is forced, but publication order is the same. Commits that arrive while a batch is syncing form the next batch, so the batch size grows with the number of concurrent writers.

## 5.4 Checkpoints

A checkpoint writes the trees of one published generation $g$ to pages and makes it the starting point of the next recovery. It walks the roots of $g$ and writes every node that is still only in memory, children before parents. Each written node is stamped with its stored reference and the checkpoint's *epoch*; when the epoch is published, the next checkpoint reuses these pages and does not visit the subtrees below them. A node that many commits changed between two checkpoints is written once. The checkpoint then forces the data files and the page directory, updates the page reference counts (Section 6), journals them, and publishes a catalog image that records $g$'s roots, the segment table and the checkpoint LSN, the end of $g$'s `Commit` frame. The image is written to a temporary file, forced, renamed and the directory forced. Last, the log is truncated before the checkpoint LSN.

Commits continue during all of this. The log is forced outside the lock that appends to it, so a commit appending its records never waits for the checkpoint's sync. By the time the checkpoint publishes, *head* may be several generations past $g$ and still refer to the in-memory nodes that the checkpoint wrote. The checkpoint swaps every node it wrote in *head* for its stored reference, copying only the parents whose children changed. It does this in two passes. The first runs without any lock and remembers its result for every node it visits. The second runs under the commit lock over the newest *head*, reuses those results and only walks what commits added in between, so the commit lock is held for the part of the tree that changed during the first pass. A transaction that started before the checkpoint still compares equal: an in-memory node stamped by a published epoch is treated as the same tree as the reference it was written as, so a finished checkpoint never pushes a transaction off the fast path.

The heap held by unwritten nodes is capped (64 MiB by default), and a checkpoint starts when half of the cap is used. It also starts when the log or the pages written since the last checkpoint pass a threshold, or when more than a million reference counts have changed. By default the threshold is a quarter of the live bytes in the data segments, but at least 2 MiB and at most 256 MiB, so that the log a crash leaves to replay stays small relative to the store while a large store does not checkpoint more often than a fixed 256 MiB would. A database can fix the threshold with the statement `CHECKPOINT AT 64MB`, which is stored in the database, and `CHECKPOINT AT AUTO` returns to the default. A background thread checks these conditions every 500 ms, and a commit that pushes the log past the threshold wakes it at once. A commit that finds the cap reached waits for a checkpoint to make room.

## 5.5 Recovery

Opening a database (Figure 4) loads the newest valid catalog image, cuts the log's last segment at the end of its last valid frame, and reads the log from the image's checkpoint LSN. Records are grouped by transaction id. For each `Commit` frame in log order whose generation is newer than the image, recovery first restores the directory entries of the pages the commit references, then checks that every page referenced by a `PageRef` reads back with a valid header and the logged checksum, and then applies the commit's `Changes` to the trees of the previous generation. The replayed generations live in memory, exactly as they did before the crash, and the checkpoint that ends every open writes them.

Two rules make the recovered state a prefix of the commit order. First, a commit is part of the recovered state only if its `Commit` frame is in the log; records of transactions that never committed are dropped. Second, if one commit's pages fail verification, it and every later commit are dropped. The second rule matters after a power failure: the operating system may write the log to disk before the data pages it refers to, and a later commit's key changes apply to the trees of the commit before it, so replaying it on top of a lost commit would produce a state that never existed. Only commits that wrote pages can hit this rule, and none of them had

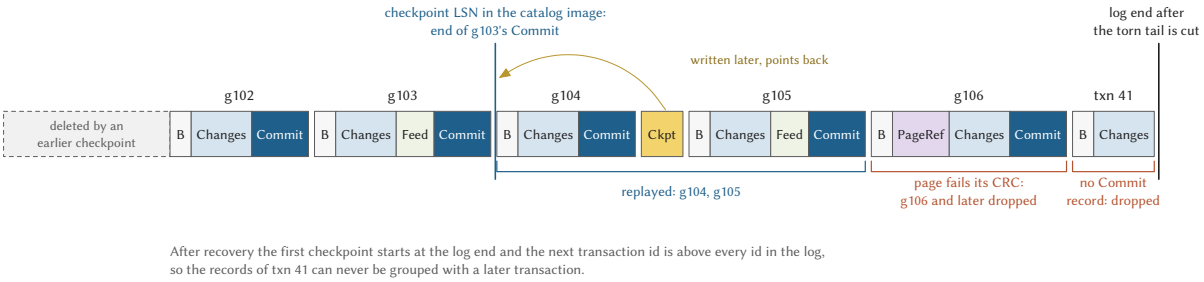



**Figure 4: Recovery reads the log from the checkpoint LSN recorded in the newest catalog image. It replays every commit whose `Commit` frame is valid, stops at the first commit whose referenced pages do not verify, and drops the records of transactions that never committed.**

been acknowledged, because acknowledgement comes after both syncs.

Recovery also guards against reusing what the log still holds. Transaction ids are persisted only in `Commit` records, so the id of a transaction that died during its commit would otherwise be handed out again while its records were still in the log, and a second crash would let recovery group the dead records with the new transaction's. Recovery therefore sets the next transaction id above every id it saw in the log, and the checkpoint at the end of the open starts from the end of the log rather than from the old checkpoint LSN, so those records are never read again.

Recovery writes nothing to the log, and the only state it leaves behind is published by the checkpoint at the end of the open, so a crash during recovery repeats the same work on the next open. History is not part of the catalog image. After a restart, time travel reaches the checkpointed generation and the generations replayed after it, and older states are available only through the change feed.

## 6 RECLAIMING SPACE

Copy-on-write never overwrites a page, so every checkpoint leaves behind the images it replaced. They stay reachable while a retained generation refers to them and become garbage afterwards. Segments are append-only, so garbage is never reused piece by piece: HStore copies the live images out of mostly dead segments and deletes the files. This needs two things, knowing which images are live and moving them without touching the trees that point at them.

### 6.1 Reference counts between checkpoints

Every page number has a reference count in a memory-mapped file laid out like the directory. The count is the number of places that point at the page in the last checkpointed generation: a slot root, a child reference in a branch, or a nested tree's root inside a record. Inline trees have no page and are never counted. A negative entry marks a page that died in generation $d$ and is waiting until no reader can see a generation older than $d$.

Counts change only at checkpoints, and only between two generations whose trees are fully written: the one the previous checkpoint wrote and the one the current checkpoint just wrote. Generations published in between are never counted. Their new nodes have no pages, and every page they reach was counted by the previous checkpoint or is *provisional* (spilled by a bulk load or written by a large commit, and resolved by the checkpoint that covers it). For each branch whose roots changed, the counter first adds one to every new root, recursing into a page only when its count rises from zero, and then subtracts one from every old root, recursing only when a count reaches zero. When a node is replaced, its children are added by the new copy and subtracted by the old one, so they cancel and are never touched. The work is proportional to the pages that changed between the two checkpoints, not to the number of commits or the size of the trees. Most of the nodes it reads are in the node cache because the checkpoint has just written them. A header flag records whether a leaf refers to any page, so leaves that hold only plain records or inline trees, such as most catalog leaves, are never decoded.

A page that died in generation $d$ is released at a checkpoint once $d$ is no newer than the oldest generation anyone can still read, the older of the oldest retained and the oldest pinned generation. Releasing clears its directory entry and subtracts its bytes from its segment's live total. The changed counts and the cleared entries are journaled with absolute values before the catalog image is published and applied afterwards, so a crash leaves the files as of one checkpoint or the next, never a mix, and applying a journal twice is harmless.

### 6.2 Compaction by moving

A sealed segment whose live bytes fall below a threshold (half of the bytes written to it, by default) is a compaction victim. Compaction reads through the victim one 64-byte unit at a time (Figure 5). Where a header starts and the directory entry for that header's page number points at exactly this place, the image is live: it is verified and its bytes are appended to the active segment. Anything else, such as a released image or an older copy of a moved one, is skipped. The header holds the page number, not the address, so neither the header nor the checksum changes. After every 4 MiB of copies the segments are forced, and only then are the directory entries pointed at the copies, so an entry never reaches the disk ahead of the bytes it points to. The victim is then retired, and its file is deleted after the next checkpoint has made the new entries durable and once no pinned reader started before the move.

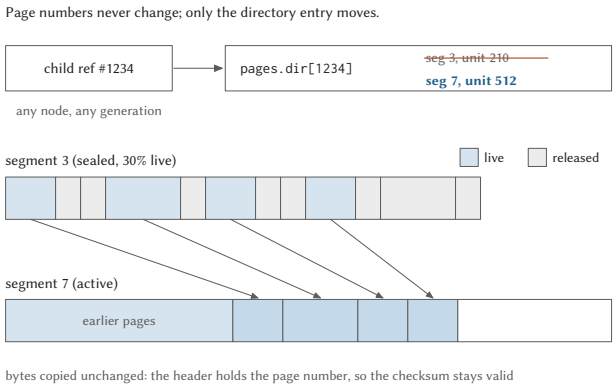



**Figure 5: Compaction copies the live images of a mostly dead segment to the active segment and changes their directory entries. Every tree, in every generation and branch, still refers to the same page numbers.**

Nothing in this takes the commit lock or commits anything. Commits continue while pages move, cached nodes stay valid because they are keyed by page number, and old generations reach the moved pages through the same directory, so compaction never shortens the time-travel window. Moves are not logged. If the engine crashes before the checkpoint that follows a compaction, recovery restores the directory entries of the replayed commits to their original addresses, which are still intact because victims are deleted only after that checkpoint. The copies become unreferenced space that a later pass reclaims. The work is proportional to the size of the victims, and the only memory it needs is the list of pages moved in the current 4 MiB.

A background pass compacts the emptiest victim every 500 ms if one exists; an explicit request compacts all of them. A new segment gets 8 MiB, doubled as long as it stays within an eighth of the bytes in the other segments, up to 256 MiB. A small database therefore has several sealed segments for compaction to choose from, and a large one has few files. The engine keeps at most 256 segment files open and closes the least recently used one, after forcing it if it has unsynced writes, so the number of open files does not grow with the database.

## 7 IMPLEMENTATION

HStore is written in Java 25 as three modules. The storage engine (about 12,000 non-blank lines) depends only on the Java base module. The database layer (about 7,900 lines) adds schemas and typed properties, property indexes, tenants and users, evidence and valid-time queries, a three-way branch merge, an embedding index fed from the change feed, higher-order operators over incidence sets, and HQL, a query language with a cost-based planner. The server (about 2,600 lines) provides a command-line tool, a line-oriented TCP protocol and a web console. The same code runs on the JVM and as a GraalVM native image, which is how the server ships in its container image.

The engine leans on recent additions to the Java platform. Segment maps and the page directory use the foreign memory API; each segment map has its own shared arena and is closed as soon as the segment is remapped or deleted, so the disk space of a deleted segment comes back at once instead of when the garbage collector finds the mapping. A per-query I/O meter, which enforces a page budget per query, is bound with scoped values rather than thread-locals. Server connections and feed subscribers run on virtual threads. Commits can therefore be issued from thousands of threads, and the group committer runs its sync on whichever committing thread leads the batch.

A Java thread that is interrupted during a read or write on a `FileChannel` closes the channel for every thread. Before this was handled, one request that timed out during a commit closed the log and every later commit failed. All shared files are therefore accessed through a wrapper that clears the interrupt flag for the length of the call, restores it afterwards, and reopens the file and retries if the channel was closed anyway; every read and write is positional, so a retry touches the same bytes.

*Testing*. The engine's tests inject a crash at each of fourteen named points: the seven steps of a commit from the first page write to publication, two steps of compaction, four of a checkpoint and the force of the log. Each commit crash point is run with both log modes; the test asserts that the reopened store holds either the state before the commit or the state after it, never a mix, and that the forward and reverse indexes agree. A second set of tests models a power failure instead of a process crash. Every file the engine writes is opened through one wrapper, which lets a test record each write, each sync of a file or a memory map and each sync of a directory, cut the power at any of these events, and rebuild the directory as a disk could have kept it: only what was synced, the last unsynced write torn in half or with a flipped byte, or a random mix, with files whose directory was never synced possibly missing. The tests then check that the recovered commits are a prefix of the attempted ones, that every commit acknowledged under synchronous durability survived, that no aborted work appears and that the change feed holds exactly one event per recovered commit. They found five bugs, all of them in files outside the log: a format marker written without a sync, a segment directory that was never synced after a file was created in it, a page directory header that could keep an unsynced first page number, and two ways in which zeroed or damaged change-feed frames were read as valid. After random workloads, the suites replay the log against the generations in memory and compare them entry by entry, and a verifier checks every reachable page (checksums, key order, balance, summaries and reference counts), which the `hstore check` command also runs. Finally, we kill the native server with SIGKILL during a bulk load, reopen it and check the counts.

## 8 EVALUATION

We compare HStore with HyperGraphDB, the only other embedded store we know of whose atoms are hyperedges with a variable number of members. The evaluation asks four questions. How do the two systems compare on reads, writes and space when the data fits in the cache and when it does not (Sections 8.2 and 8.4)? What do commits and reads cost one at a time, and how do reads behave while a writer is committing (Section 8.3)? How do the systems recover from a crash and reclaim space after deletes (Section 8.5)?

**Table 1: Workloads, in execution order.**

| Workload | Operation |
|---|---|
| ingest.nodes, .edges | insert all nodes, then all hyperedges, 1,000 per transaction |
| read.incidence | list the hyperedges that contain a node; 100,000 probes |
| read.members | list a hyperedge's members; 100,000 probes |
| read.comembership | count the hyperedges that contain both nodes of a pair |
| read.twohop | count distinct nodes that share a hyperedge with a node |
| read.incidence.parallel | read.incidence from 8 threads |
| latency.read | one incidence lookup per read transaction, each timed |
| write.update | replace a node's value; 20,000 updates, 1,000 per transaction |
| latency.commit | one update per transaction, each timed; 5,000 commits |
| mixed.read, .write | 7 reader threads for 10 s beside a writer committing 10 updates per transaction on a fixed schedule of 1,000 updates/s |
| large.* | create one hyperedge that contains every node, scan it 400 times, probe it 100,000 times |
| reopen | close cleanly and open again |
| read.incidence.cold | read.incidence right after the reopen |
| churn.delete, .remove | delete 20% of the hyperedges; remove one member from 20,000 others |
| compaction | single-update commits at 50/s while a full space reclamation runs |
| disk, .churned, .compacted | bytes on disk after load, after the deletes, after reclamation |
| recover | open a store whose ingest process was killed with SIGKILL |

And how much does each of HStore's main design decisions contribute (Section 8.6)?

## 8.1 Setup

*Dataset.* The dataset is generated from a fixed seed, so both systems load the same hypergraph. At scale $s$ it has 50,000 $s$ nodes and 100,000 $s$ hyperedges. A hyperedge starts with two members and gains each further member with probability 0.62, up to 32, for a mean cardinality of 3.6. Members are drawn as $\lfloor nu^{2.2} \rfloor$ for uniform $u \in [0, 1)$ over $n$ nodes, so low-numbered nodes become hubs. At scale 1 half of the nodes are in at most 5 hyperedges, 370 are in more than 48, and the largest is in 2,645; at scale 4 the largest is in 5,714. Probe sets from the same seed select 100,000 random atoms, 20,000 pairs of nodes that share a hyperedge, and 20,000 node updates. A churn set selects 20% of the hyperedges for deletion and 20,000 other hyperedges, each losing one member.

*Workloads.* Table 1 lists the 26 workloads, which run in this order in one process per system and run. Each adapter uses the plain storage API of its system: long ids and `forEachIncidentEdge`, `forEachMemberId` and `Writer.set` for HStore, persistent handles and `getIncidenceSet`, `getTargetAt` and `graph.replace` for HyperGraphDB. We do not use HyperGraphDB's traversal framework, so that both sides measure storage. HyperGraphDB links are immutable, so removing a member replaces the link. Every read and delete workload returns a checksum, such as the number of ids visited, and both systems returned the same checksum in every run.

*Measurement.* Throughput is the operation count divided by the elapsed time of the whole operation set. Each write workload ends with a flush outside the timed section (an HStore checkpoint or a JE `sync`), so bytes still in buffers are charged to the workload that produced them. Every read workload except read.incidence.cold first runs untimed for at least 2 s, so both systems are measured with their read paths compiled by the JIT. The latency workloads time every transaction; in mixed and compaction the writer follows a fixed schedule and its latency is measured from the scheduled start, so a stall is charged to every commit queued behind it [24]. Both systems count their own file I/O: HStore its log and page bytes, including pages moved by compaction, and HyperGraphDB the bytes JE writes and reads, including its cleaner. Allocation per operation comes from the JVM's per-thread allocation counters and the retained heap from a full collection after ingest.

*Configurations.* We run three configurations. S1 async is scale 1 with asynchronous commits. S1 sync is scale 1 with synchronous commits. S4 async is scale 4 with asynchronous commits; there the files of both systems are larger than their 64 MiB caches. Both caches get the same 64 MiB budget. HStore's covers decoded nodes; JE's covers its B+tree nodes and records, and HyperGraphDB's atom cache comes on top and cannot be capped in bytes. HyperGraphDB 1.4 begins every JE transaction with `WRITE_NO_SYNC`, which overrides the environment's `COMMIT_SYNC`; under S1 sync its adapter therefore forces JE's log after every commit, which is what `COMMIT_SYNC` would do.

*Machine.* All runs used one Azure `Standard_D8ds_v5` virtual machine: four cores (eight hardware threads) of an Intel Xeon Platinum 8370C at 2.8 GHz, 32 GiB of memory, and Ubuntu 24.04 with Linux 6.17. Both systems kept their files on the machine's local 300 GB temporary SSD, formatted as ext4, and ran on Oracle GraalVM 25.0.4 in JVM mode. HyperGraphDB is version 1.4 built from commit `99485a1` with Berkeley DB JE 5.0.73, the JE version its storage module is written against. Each system runs in its own JVM with a fixed 4 GiB heap and the Parallel collector. Runs alternate between the two systems, so that a slow period on the machine affects both. We report medians over 5, 5 and 3 runs per system for the three configurations. A ratio is oriented so that values above 1 favour HStore, and it is marked with † when the ranges of the two systems' runs overlap.

## 8.2 Throughput

Table 2 lists every median and Figure 6 plots the ratios. Every read and delete workload returned the same checksum on both systems in every run. HStore is ahead on every workload whose rate the harness does not fix, with one exception: single-update commits under asynchronous durability at scale 1, where the two systems are within each other's range ($0.95^{\dagger}$).

*Reads.* A batch of incidence lookups is 2.5 times faster at scale 1 and 2.4 times faster at scale 4, and member lookups are 1.4 and 1.7 times faster. Counting the hyperedges two nodes share is 5.5 times faster at scale 1. HStore answers it by intersecting the two incidence trees and skipping every pair of subtrees whose key ranges do not overlap; HyperGraphDB evaluates the conjunction of two incidence conditions. From eight threads HStore reaches 14.1 M incidence lookups per second at scale 1, while HyperGraphDB stays at 1.28 M, about its single-thread rate of 1.25 M. Right after a reopen the gap is widest: 25.6 times at scale 1 and 81.6 times at scale 4.

**Table 2: Median results. Rates are operations per second; disk rows are bytes on disk; reopen and recover are times. A ratio above 1 favours HStore, and † marks a ratio whose run ranges overlap. The S1 sync column gives only the ratio; its rates differ from S1 async only for the commit-bound workloads discussed in the text.**

| | S1 async | | | S1 sync | S4 async | | |
|---|---|---|---|---|---|---|---|
| Workload | HStore | HyperGraphDB | ratio | ratio | HStore | HyperGraphDB | ratio |
| ingest.nodes | 33.3 k | 19.6 k | 1.7 | 1.7 | 42.9 k | 24.0 k | 1.8 |
| ingest.edges | 28.9 k | 21.3 k | 1.4 | 1.3 | 21.5 k | 2.71 k | 7.9 |
| read.incidence | 3.10 M | 1.25 M | 2.5 | 3.1 | 2.62 M | 1.11 M | 2.4 |
| read.members | 2.29 M | 1.66 M | 1.4 | 1.6 | 2.13 M | 1.27 M | 1.7 |
| read.comembership | 272 k | 49.8 k | 5.5 | 4.9 | 281 k | 41.4 k | 6.8 |
| read.twohop | 243 k | 83.6 k | 2.9 | 3.4 | 172 k | 80.9 k | 2.1 |
| read.incidence.parallel | 14.1 M | 1.28 M | 11 | 12.8 | 8.21 M | 1.26 M | 6.5 |
| latency.read | 2.07 M | 505 k | 4.1 | 3.8 | 1.79 M | 470 k | 3.8 |
| write.update | 40.9 k | 7.26 k | 5.6 | 5.8 | 30.2 k | 2.47 k | 12.2 |
| latency.commit | 6.96 k | 7.33 k | 0.95† | 1.2 | 10.1 k | 2.54 k | 4.0 |
| mixed.read | 8.73 M | 1.13 M | 7.7 | 8.0 | 8.55 M | 1.11 M | 7.7 |
| mixed.write | 1,000 | 1,000 | 1.0 | 1.0 | 1,000 | 999 | 1.0 |
| large.ingest | 431 k | 47.6 k | 9.1 | 7.4 | 489 k | 13.6 k | 36.1 |
| large.scan | 1,668 M | 344 M | 4.8 | 4.5 | 1,583 M | 358 M | 4.4 |
| large.probe | 6.60 M | 1.01 M | 6.5 | 7.2 | 5.61 M | 844 k | 6.7 |
| reopen | 78.5 ms | 978 ms | 12.5 | 13 | 92.9 ms | 1,279 ms | 13.8 |
| read.incidence.cold | 1.73 M | 67.6 k | 25.6 | 28.1 | 1.23 M | 15.1 k | 81.6 |
| disk | 85.7 MiB | 400 MiB | 4.7 | 4.3 | 172 MiB | 2.30 GiB | 13.7 |
| churn.delete | 16.1 k | 5.18 k | 3.1 | 2.9 | 2.64 k | 1.99 k | 1.3 |
| churn.remove | 23.1 k | 10.7 k | 2.2 | 2.1 | 4.08 k | 3.53 k | 1.2 |
| read.incidence.churned | 3.10 M | 1.30 M | 2.4 | 2.6 | 2.75 M | 1.05 M | 2.6 |
| disk.churned | 116 MiB | 293 MiB | 2.5 | 2.0 | 1.14 GiB | 1.91 GiB | 1.7 |
| compaction.baseline | 50.1 | 50.1 | 1.0† | 1.0† | 50.1 | 50.1 | 1.0 |
| compaction | 50.1 | 50.1 | 1.0† | 1.0† | 50.1 | 50.0 | 1.0 |
| disk.compacted | 93.3 MiB | 234 MiB | 2.5 | 2.7 | 183 MiB | 1.15 GiB | 6.4 |
| recover | 84.8 ms | 113 ms | 1.3† | 1.4 | 162 ms | 449 ms | 2.8 |

HStore reads its segments through memory maps that the operating system's page cache still holds and decodes a leaf record only when it is read, while JE fills its own cache from its log one node at a time.

*Writes.* Node and hyperedge ingest are 1.7 and 1.4 times faster at scale 1. At scale 4, where both systems' files outgrow their 64 MiB caches, hyperedge ingest is 7.9 times faster, because HyperGraphDB drops from 21.3 k to 2.71 k hyperedges per second. Updates in batches of 1,000 are 5.6 and 12.2 times faster. Creating one hyperedge that holds every node is 9.1 times faster at scale 1 and 36.1 times at scale 4, and scanning it is 4.8 times faster. Deletes are the closest. At scale 4 HStore deletes 2.64 k hyperedges per second, 1.3 times HyperGraphDB's rate, against 3.1 times at scale 1. Deleting a hyperedge removes it from the reverse index of every member, and at scale 4 those postings are spread over many more pages than the cache holds.

*Opening a database.* A clean reopen takes 78.5 ms against 978 ms at scale 1 (12.5 times faster). HStore loads the catalog image and maps the segments; nothing is replayed after a clean shutdown.

## 8.3 Latency

Figure 7 shows the percentiles. A read transaction that looks up one incidence set takes 0.429 µs at the median on HStore and 1.55 µs on HyperGraphDB, and 1.23 µs against 12.1 µs at the 99.9th percentile.

Single-update commits are where the systems are closest. Under asynchronous durability at scale 1, HyperGraphDB's median commit is faster (93.7 µs against 143 µs), but its tail is longer: 767 µs against 228 µs at the 99th percentile and 1.78 ms against 338 µs at the 99.9th. Under synchronous durability every commit waits for an fsync, and HStore's median is 537 µs against 628 µs; a commit writes 659 B to HStore's log and data files and 57.5 KiB to JE's. At scale 4 HStore's commits are faster at every percentile up to the 99.9th (287 µs against 6.44 ms), but its slowest commit in a run is slower: 34.9 ms at the median of the runs against 9.31 ms. We have not yet traced these single stalls.

The mixed workload runs seven readers for 10 s beside a writer that commits 1,000 updates per second on a fixed schedule. Both writers kept the schedule. HStore's readers complete 7.7 times as many lookups, with a median latency of 0.649 µs against 3.72 µs and a 99.9th percentile of 5.47 µs against 94.1 µs. HStore readers never take a lock, so the writer does not slow them, and the writer in turn finishes its transactions sooner: its 99th percentile, measured from the scheduled start, is 5.68 ms against 28.5 ms.

## 8.4 Resources

*Bytes written.* Logging keys instead of pages shows most clearly in the bytes written per change (Figure 8a). An update in a batch writes 224 B on HStore and 9.12 KiB on HyperGraphDB, and a single-update commit 659 B against 11.1 KiB. At scale 4 hyperedge ingest writes 1.04 KiB per hyperedge against 29.5 KiB. Over a whole run at scale 4, HStore wrote 2.06 GiB and HyperGraphDB

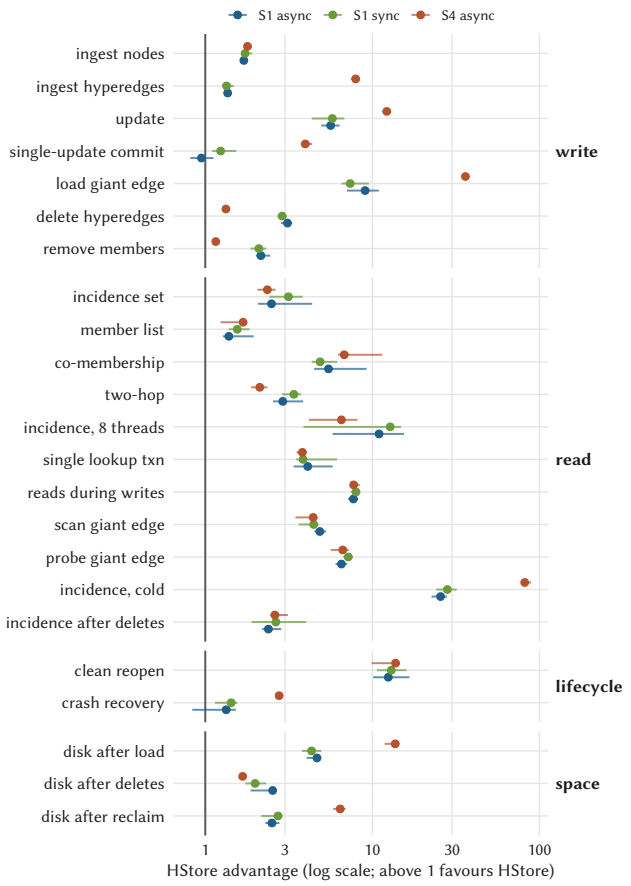



**Figure 6: HStore's advantage per workload and configuration: the ratio of the medians, with a bar from the least to the most favourable pairing of runs. Workloads with a fixed rate are left out.**

104 GiB, 83.0 GiB of it during the compaction workload, while JE's cleaner copied live records out of old log files after the deletes.

*Allocation and heap.* Reads on HStore allocate almost nothing (Figure 8b): 1 B per incidence lookup in a batch against 116 B, and 49 B per single-lookup transaction against 4.77 KiB. In the mixed workload, where the readers allocate 50 B and 1.90 KiB per lookup, the collector spent 57 ms on HStore and 241 ms on HyperGraphDB over the 10 s. HStore keeps more heap after ingest, 179 MiB against 118 MiB at scale 1 and 391 MiB against 303 MiB at scale 4. Both its node cache and its write buffer of nodes that no checkpoint has written yet are on the heap.

## 8.5 Space and recovery

After the load HStore's files take 85.7 MiB at scale 1 and 172 MiB at scale 4, against 400 MiB and 2.30 GiB for HyperGraphDB (Figure 8c). The deletes grow HStore's files to 1.14 GiB at scale 4, because a page that a delete replaced stays until a checkpoint releases it and compaction empties its segment. HStore gives that space back on its own. The compaction workload starts 10 s later, after ten seconds of single-update commits with the background compactor running, and by then the files are back to 182 MiB; the explicit compaction that follows takes 11 ms. JE's cleaner, run until it reports nothing left to clean, takes 13 min at scale 4. At scale 1 it usually takes well under a second, but one of the five runs took 13 min. Commits continued on both systems while space was reclaimed and kept their schedule of 50 per second. After reclamation HStore's files are 93.3 MiB and 183 MiB, and HyperGraphDB's 234 MiB and 1.15 GiB.

In the recover workload a child process ingests nodes in transactions of 1,000 and is killed with SIGKILL once half of them are acknowledged. No run of either system lost an acknowledged commit. Opening the killed store takes 84.8 ms on HStore and 113 ms on HyperGraphDB at scale 1, and 162 ms against 449 ms at scale 4. At scale 1 the child writes only a few megabytes of log before it is killed, so with the automatic threshold of Section 5 the open replays only the log written since the last background checkpoint.

## 8.6 What each design choice contributes

To separate three of the decisions in Sections 4 and 5, we ran HStore alone at the commit that introduced each of them and at the commit just before it, with 3 runs of each build on the same machine (Figure 9). The two builds of a pair differ only in that change. Each pair comes from earlier in the project's history than the version measured above, so its absolute numbers are not comparable with Table 2.

*Logging keys instead of pages.* Before this change a commit wrote every new node from each changed leaf to the root. An update in a batch wrote 1.85 KiB before it and 221 B after it, and a single-update commit 14.9 KiB and 649 B. Batched updates became 1.4 times faster and single-update commits 1.8 times faster. The cost is recovery. The open now replays the logged keys into the trees instead of finding pages already written, and opening the killed store took 105 ms instead of 37.5 ms. The automatic checkpoint threshold, added later, bounds how much log the open has to replay. Most read workloads were 5 to 17% slower after the change, two-hop queries excepted, and we have not investigated why.

*Inline member trees.* Storing a small membership inside the hyperedge's record makes member lookups 1.6 times faster and two-hop queries 1.3 times faster, because a lookup no longer follows a root reference to a tree of its own. It also removes a separate small leaf per hyperedge, and the files after reclamation shrink from 202 MiB to 66.1 MiB. Single-lookup read transactions were slightly slower, at 0.88 times their earlier rate.

*Records decoded on demand.* With data larger than the cache, decoding a leaf record only when it is read makes member lookups 10.3 times faster (179 k to 1.85 M per second) and two-hop queries 13.8 times faster, since a lookup that misses the cache decodes one record instead of a whole page. Incidence lookups right after a reopen gain 1.4 times. The same change cost almost half of the incidence rate from eight threads, from 11.7 M to 6.43 M lookups per second, and deletes ran at 0.85 times their earlier rate. We have not yet found what the eight readers contend on.

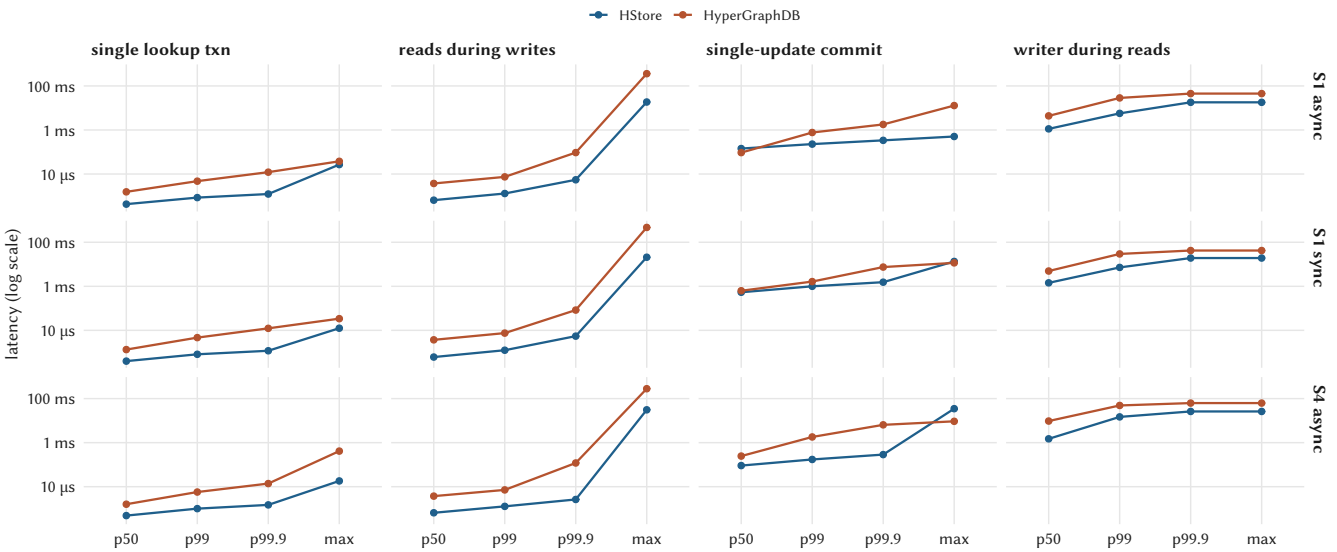



**Figure 7: Latency percentiles, medians over runs. Read latencies are per transaction with one lookup; commit latencies include the wait for the group commit; in the mixed workload the writer's latency is measured from each transaction's scheduled start.**

## 8.7 Limits of this evaluation

Both systems run on the JVM, on one machine, with one synthetic dataset. The dataset's skewed membership resembles real hypergraphs, but its hyperedges carry no properties beyond a name, and we did not test workloads with large property values or with many concurrent writers. HyperGraphDB is the only system we compare with, because it is the only embedded store we know of with first-class hyperedges, and its storage layer, JE 5.0.73, is more than a decade old. A comparison with a graph database storing reified hyperedges would answer a different question and is left for future work.

# 9 RELATED WORK

*Hypergraph and n-ary data stores.* HyperGraphDB [14] is the closest system and our baseline. Its atoms are identified by UUID handles, a link is an ordered array of target handles, and an index maps every atom to its incidence set; storage is delegated to Berkeley DB Java Edition. A link is immutable, so changing a membership replaces the link, and the incidence index has to be updated for every member. OpenCog's AtomSpace [11] stores typed atoms and links in memory for reasoning workloads. TypeDB [9] gives *n*-ary relations and their role players a type system and a query language, and stores them in a key-value engine. Property-graph databases and RDF stores represent *n*-ary relations by reification [1], and Wikidata attaches qualifiers to statements for the same purpose [30]. HStore differs from all of them in that a hyperedge's membership is a storage structure of its own, versioned, counted and summarised like every other tree in the engine.

*Hypergraph processing.* Hygra [28], MESH [12] and HyperX [13] run analytics over hypergraphs held in memory, on one machine or a cluster. They assume a static input and optimise traversal and message passing. HStore is a storage engine for data that changes, and the analytic operators in its database layer read from its snapshots.

*Copy-on-write and shadow paging.* Shadow paging goes back to Lorie [18]. Rodeh showed how to make B-trees copy-on-write with reference counts so that snapshots and clones are cheap [25], and Btrfs builds a file system on that design [26]. LMDB [5] uses a copy-on-write B+tree with a single writer and readers that pin a root, much like HStore's generations. These systems write the changed path at every commit. HStore defers those writes to checkpoints and logs keys instead, which is closer to how a log-structured store absorbs small writes, while keeping the property that a published version never changes. Making data structures persistent in the functional sense [10] and the owner tokens that let one transaction change its own fresh nodes in place follow the same idea at the level of single objects.

*Logging, recovery and log-structured storage.* ARIES [19] logs page-level changes and redoes them in place. HStore never changes a page in place, so it logs logical key changes and replays them onto the last checkpoint's trees, and its only physical log records are page references for the rare commits that write pages. The rule that recovery stops at the first commit whose pages are missing addresses the reordering of writes by the file system that crash-consistency studies have catalogued [23, 31]. Log-structured file systems [27] and LSM-trees [21] append all writes and reclaim space by cleaning or merging; Berkeley DB Java Edition, under HyperGraphDB, is a log-structured B+tree whose cleaner copies live records forward. HStore's compaction also copies live data forward, but because trees refer to pages by number through a directory, it copies pages without rewriting their parents. The Bw-tree [17] uses a mapping table from logical page ids to physical addresses for a related reason, to install delta updates and relocate pages without changing references. LeanStore [16] replaces the

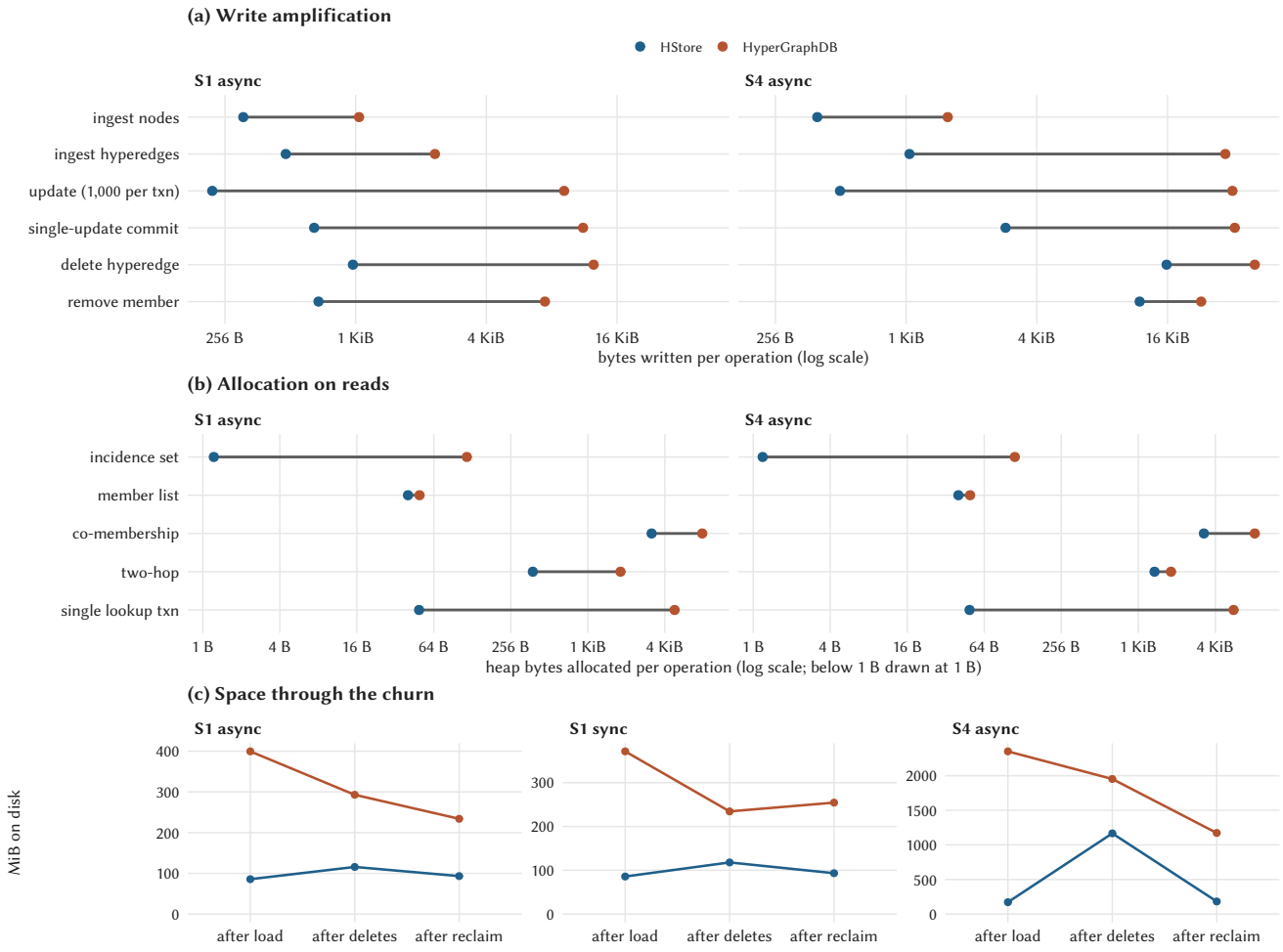



**Figure 8: (a) Bytes written to the log and data files per operation, including JE's cleaner and HStore's compaction. (b) Heap bytes allocated per operation on the read workloads, over all threads. (c) Bytes on disk after the load, after the deletes and after reclamation, each measured after a clean shutdown.**

buffer pool's hash table with pointer swizzling; HStore's node cache stays a separate, budgeted table because its nodes are immutable and shared by every reader.

*Order maintenance and set algebra.* Ordered hyperedges use the order-maintenance labelling of Dietz and Sleator and its later simplifications [3, 8], as ORDPATH labels do for XML [22]. Intersections use a leapfrog join [29]. Additive multiset hashes [6] and polynomial fingerprints [15] give each tree a content fingerprint that is independent of its shape.

## 10 CONCLUSION

HStore stores a hypergraph as trees of a single kind. Because every tree is a counted, summarized copy-on-write B+tree, the same code answers cardinality, rank, overlap and difference questions for a hyperedge's members, an atom's incidence set and the catalog, and the same checkpoint, recovery and compaction machinery serves all of them. Logging keys instead of pages removes most of the write cost that copy-on-write usually carries, and addressing pages through a directory lets compaction work without touching the trees. Against HyperGraphDB the result is faster reads and writes, fewer bytes written and less space on disk, at the price of more heap.

*Limitations.* A database is opened by one process, and there is no replication. Commits are validated and appended to the log under one lock, so commit throughput is bounded by that short critical section even when writers change different trees. Retained history lives in memory, so after a restart time travel reaches back only to the last checkpoint and the commits replayed after it. Unwritten nodes and decoded nodes are both Java heap, and Section 8.4 shows that HStore keeps more of it than HyperGraphDB. At scale 4 the slowest single commit in a run is slower on HStore than on HyperGraphDB, and we have not yet found what causes

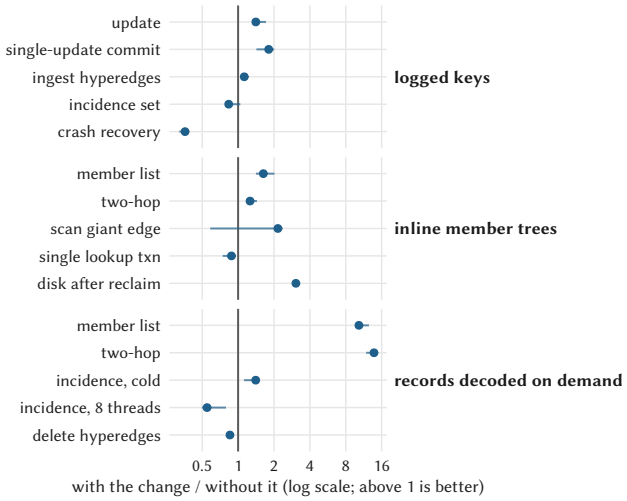



**Figure 9: Effect of three design choices, each measured on HStore alone at the commit that introduced it and at the commit before it. Logged keys and inline member trees run at scale 1, records decoded on demand at scale 4.**

these stalls. Decoding leaf records on demand roughly halved the rate of incidence lookups from eight threads at scale 4, a regression we have not yet traced either. For small databases most of the space left after deletes and compaction is the active log segment and the change feed rather than data pages, and their retention is not yet tuned for small stores.

*Future work.* We plan to compare HStore with a graph database that stores reified hyperedges, to run the evaluation with data larger than main memory, and to measure many concurrent writers on overlapping hyperedges, where member-level conflict detection should matter most.

## ACKNOWLEDGMENTS

We thank Ujjwal Jain, who found that the first checkpoint after a recovery could record a log position that came before records recovery had discarded, so that a second crash would read them again, and contributed the fix together with the guard against reusing transaction ids described in Section 5.5.

## REFERENCES

[1] Renzo Angles and Claudio Gutierrez. 2008. Survey of Graph Database Models. *Comput. Surveys* 40, 1, Article 1 (2008).

[2] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. 2020. Networks beyond Pairwise Interactions: Structure and Dynamics. *Physics Reports* 874 (2020), 1–92.

[3] Michael A. Bender, Richard Cole, Erik D. Demaine, Martin Farach-Colton, and Jack Zito. 2002. Two Simplified Algorithms for Maintaining Order in a List. In *Algorithms, ESA 2002 (Lecture Notes in Computer Science, Vol. 2461)*. 152–164.

[4] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. 1–10.

[5] Howard Chu. 2011. MDB: A Memory-Mapped Database and Backend for OpenLDAP. In *LDAPCon 2011*.

[6] Dwaine Clarke, Srinivas Devadas, Marten van Dijk, Blaise Gassend, and G. Edward Suh. 2003. Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. In *Advances in Cryptology, ASIACRYPT 2003 (Lecture Notes in Computer Science, Vol. 2894)*. 188–207.

[7] Fernando J. Corbató. 1968. *A Paging Experiment with the Multics System.* Technical Report MAC-M-384. MIT Project MAC.

[8] Paul Dietz and Daniel Sleator. 1987. Two Algorithms for Maintaining Order in a List. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing (STOC)*. 365–372.

[9] Christoph Dorn and Haikal Pribadi. 2024. TypeQL: A Type-Theoretic & Polymorphic Query Language. *Proceedings of the ACM on Management of Data* 2, 2 (2024), 1–27.

[10] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[11] David Hart and Ben Goertzel. 2008. OpenCog: A Software Framework for Integrative Artificial General Intelligence. In *Proceedings of the First Conference on Artificial General Intelligence (AGI 2008) (Frontiers in Artificial Intelligence and Applications, Vol. 171)*. IOS Press, 468–472.

[12] Benjamin Heintz, Rankyung Hong, Shivangi Singh, Gaurav Khandelwal, Corey Tesdahl, and Abhishek Chandra. 2019. MESH: A Flexible Distributed Hypergraph Processing System. In *IEEE International Conference on Cloud Engineering (IC2E)*. 12–22.

[13] Jin Huang, Rui Zhang, and Jeffrey Xu Yu. 2015. Scalable Hypergraph Learning and Processing. In *IEEE International Conference on Data Mining (ICDM)*. 775–780.

[14] Borislav Iordanov. 2010. HyperGraphDB: A Generalized Graph Database. In *Web-Age Information Management, WAIM 2010 International Workshops (Lecture Notes in Computer Science, Vol. 6185)*. Springer, 25–36.

[15] Richard M. Karp and Michael O. Rabin. 1987. Efficient Randomized Pattern-Matching Algorithms. *IBM Journal of Research and Development* 31, 2 (1987), 249–260.

[16] Viktor Leis, Michael Haubenschild, Alfons Kemper, and Thomas Neumann. 2018. LeanStore: In-Memory Data Management beyond Main Memory. In *34th IEEE International Conference on Data Engineering (ICDE)*. 185–196.

[17] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for New Hardware Platforms. In *29th IEEE International Conference on Data Engineering (ICDE)*. 302–313.

[18] Raymond A. Lorie. 1977. Physical Integrity in a Large Segmented Database. *ACM Transactions on Database Systems* 2, 1 (1977), 91–104.

[19] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. 1992. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems* 17, 1 (1992), 94–162.

[20] Michael A. Olson, Keith Bostic, and Margo Seltzer. 1999. Berkeley DB. In *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. 183–191.

[21] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica* 33, 4 (1996), 351–385.

[22] Patrick O'Neil, Elizabeth O'Neil, Shankar Pal, Istvan Cseri, Gideon Schaller, and Nigel Westbury. 2004. ORDPATHs: Insert-Friendly XML Node Labels. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. 903–908.

[23] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2014. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 433–448.

[24] Mark Raasveldt, Pedro Holanda, Tim Gubner, and Hannes Mühleisen. 2018. Fair Benchmarking Considered Difficult: Common Pitfalls in Database Performance Testing. In *Proceedings of the Workshop on Testing Database Systems (DBTest)*. Article 2.

[25] Ohad Rodeh. 2008. B-trees, Shadowing, and Clones. *ACM Transactions on Storage* 3, 4, Article 15 (2008).

[26] Ohad Rodeh, Josef Bacik, and Chris Mason. 2013. BTRFS: The Linux B-Tree Filesystem. *ACM Transactions on Storage* 9, 3, Article 9 (2013).

[27] Mendel Rosenblum and John K. Ousterhout. 1992. The Design and Implementation of a Log-Structured File System. *ACM Transactions on Computer Systems* 10, 1 (1992), 26–52.

[28] Julian Shun. 2020. Practical Parallel Hypergraph Algorithms. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 232–249.

[29] Todd L. Veldhuizen. 2014. Leapfrog Triejoin: A Simple, Worst-Case Optimal Join Algorithm. In *Proceedings of the 17th International Conference on Database Theory (ICDT)*. 96–106.

[30] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[31] Mai Zheng, Joseph Tucek, Dachuan Huang, Feng Qin, Mark Lillibridge, Elizabeth S. Yang, Bill W. Zhao, and Shashank Singh. 2014. Torturing Databases for Fun and Profit. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 449–464.